

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Quiz 1: due 2/25
  - Homework 6: due 2/28
  - Lab 2: due on 2/28
  - Project 2: due on 3/18
- Midterm exam on Thursday 3/3
  - In-person, on paper, closed book

Previous lecture …

- How does the ***short-term scheduler*** select the next process to run?
  - FCFS, SJF, Priority, RR

Muddiest Points (Scheduling)

- can a mix of scheduling algorithms be utilized by the scheduler?
- What makes the priorities algorithm significantly different from the shortest process first algorithm?
- How does First Come First Serve avoid starvation? What if a process runs indefinitely?
- What is the most commonly used in modern computer design ?
- Convoy Effect
- In some points, we use Service Time for finding out which process is short time, is there any table that has the times of all process? or cpu or user predict these times?
- non-preemptive versus pre-emptive
- First Come First Serve Scheduling and Arrival Time how they work together

Muddiest Points *(Round Robin)*

- Why does making the time quantum much longer than the CPU burst penalize I/O bound processes?
- What actually is "quantum"? Is it something that can be implemented? Or is it just conceptual
- quantum/time slice size choice
- Can the round-robin time quantum be dynamically selected?
- Round robin vs FCFS

Muddiest Points *(misc.)*

- Will practice exam be posted
- I'm from the future and Thoth went down again. Project 2 extension?
- What specific material is covered on the midterm exam? I'm somewhat unclear on that question.

Problem of the Day: CPU Scheduling

How does the ***short-term scheduler*** select the next process to run?

Multilevel Feedback Scheduling

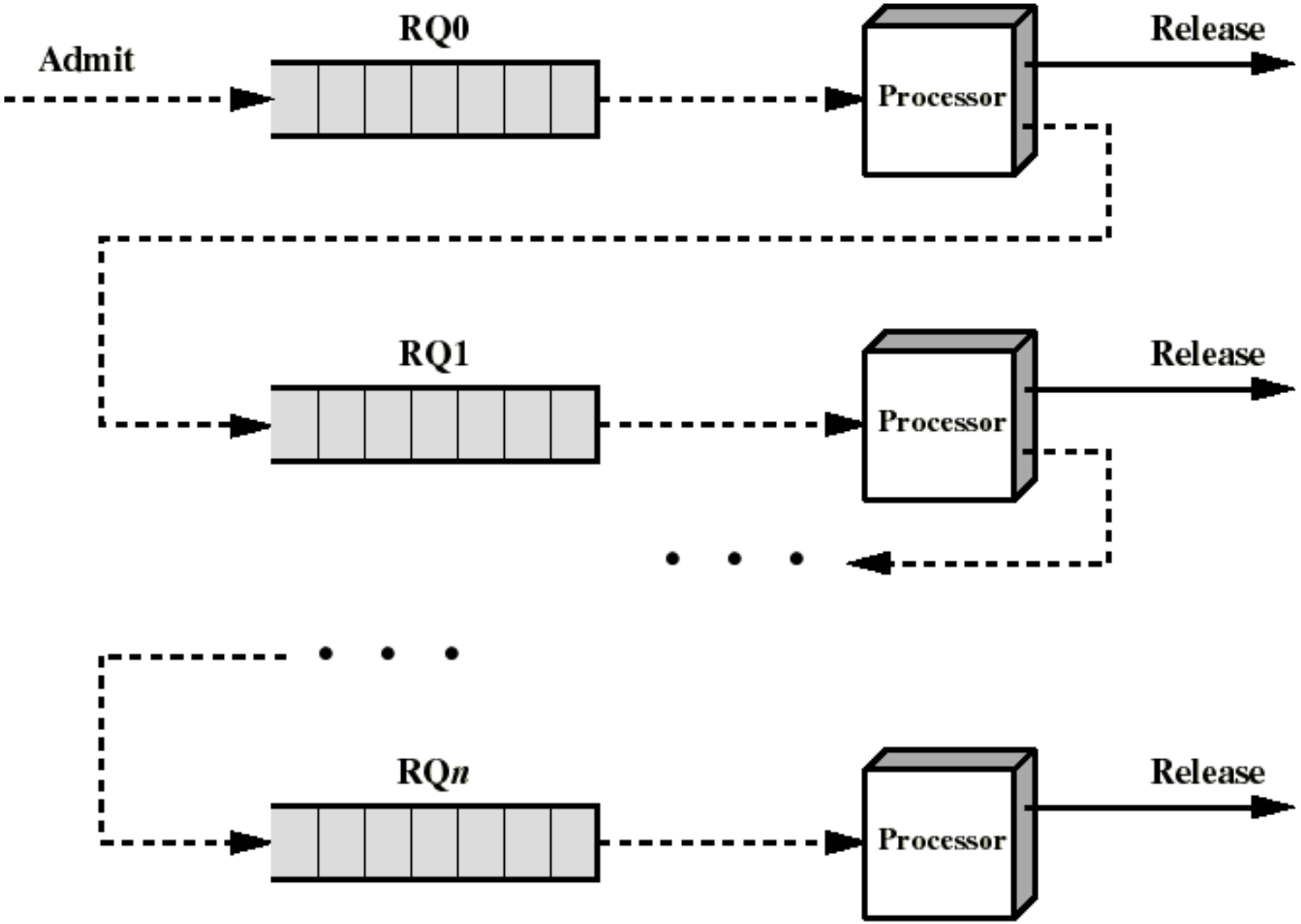
- Preemptive scheduling with dynamic priorities
- N ready to execute queues with decreasing priorities:
  - $P(RQ_0) > P(RQ_1) > ... > P(RQ_{N-1})$
- Dispatcher selects a process for execution from RQ_i only if RQ_{i-1} to RQ_0 are empty

Multilevel Feedback Scheduling

- New process are placed in RQ_0
- After the first quantum, they are moved to RQ_1, and to RQ_2 after the second quantum, … and to RQ_{N-1} after the Nth quantum
- I/O-bound processes remain in higher priority queues.
  - CPU-bound jobs drift downward.
  - Hence, long jobs may starve

Multiple Feedback Queues

Different RQs may have different quantum values



Time Quantum for feedback Scheduling

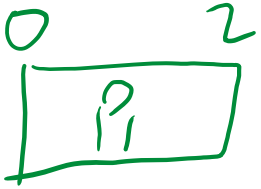
- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

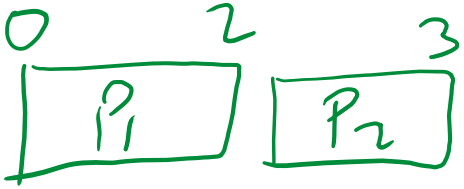
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

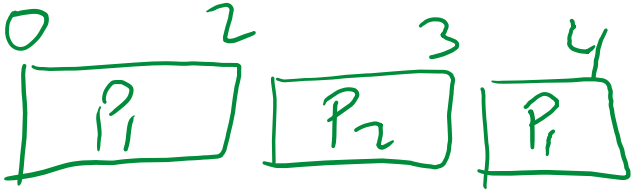
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

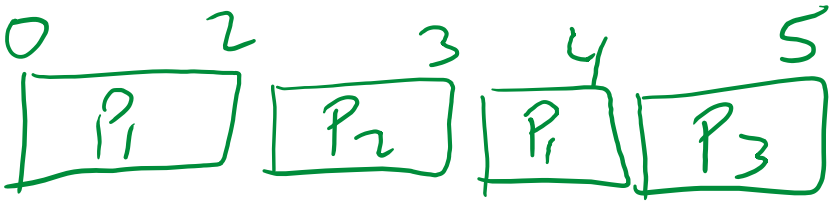
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

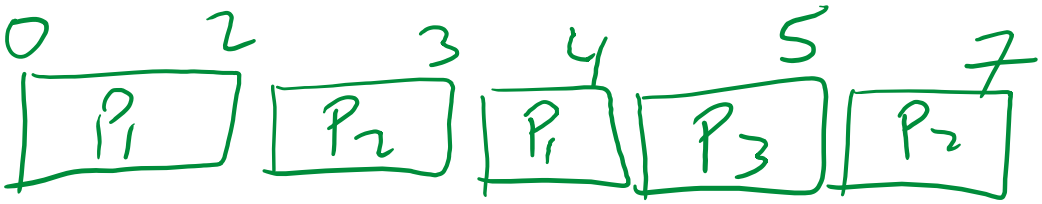
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

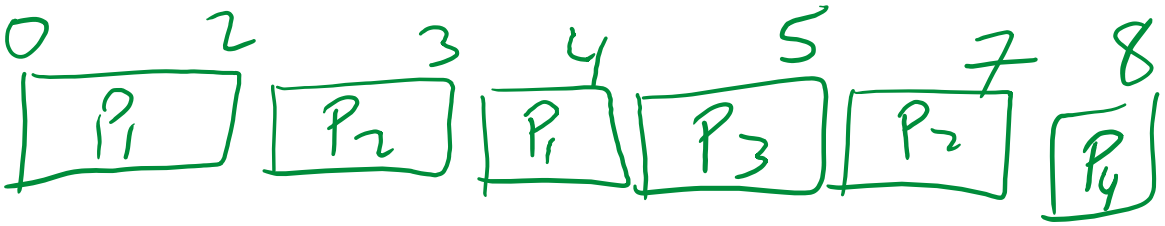
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

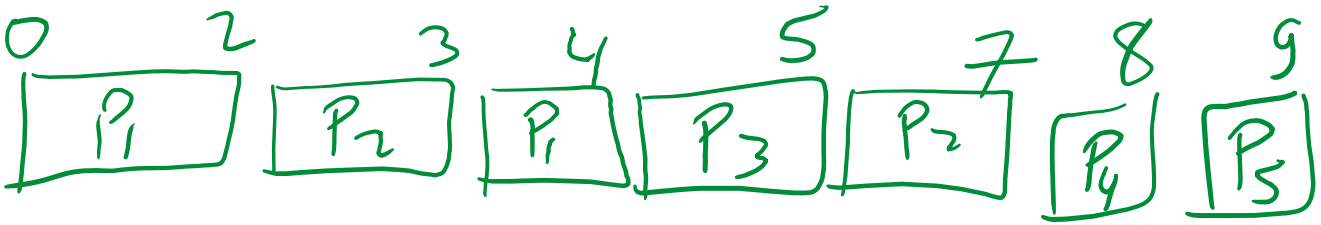
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

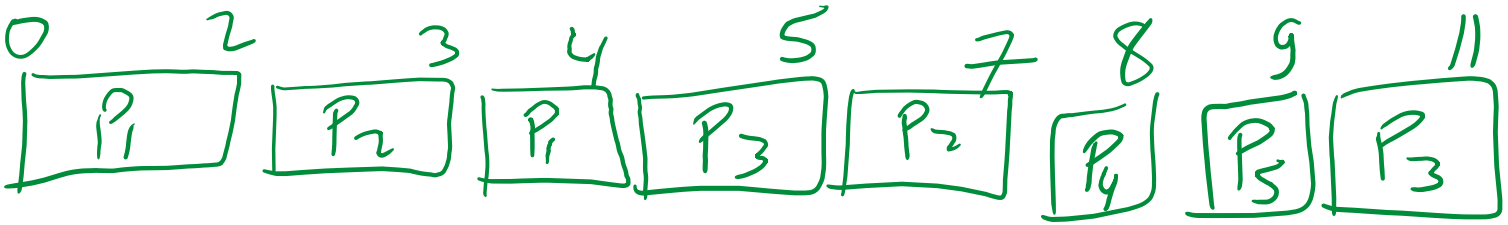
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

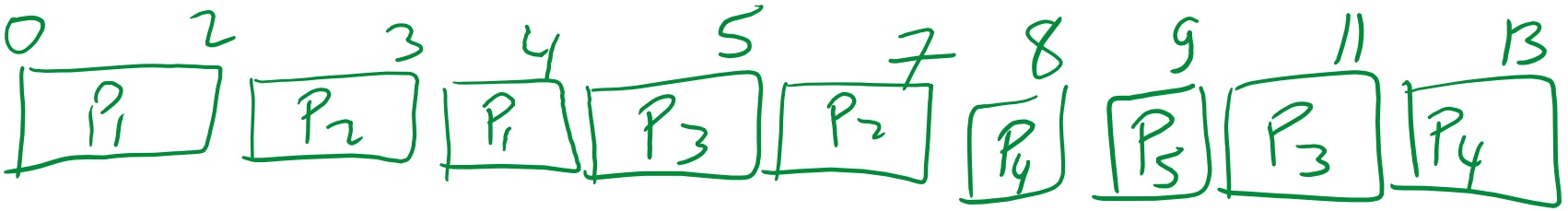
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

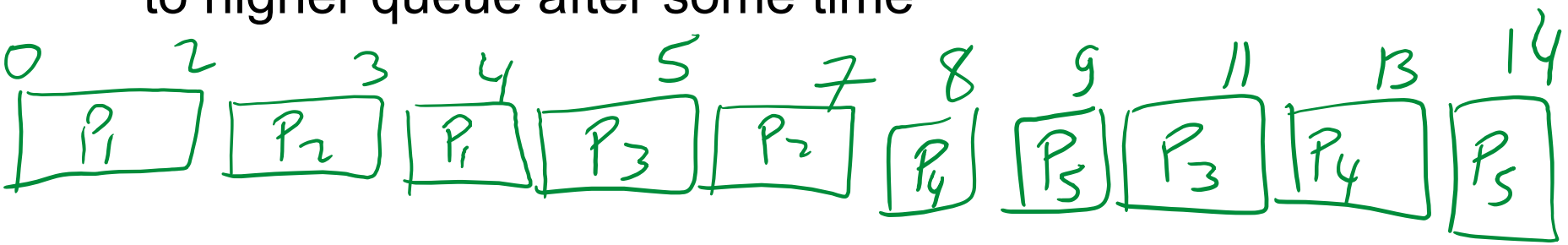
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

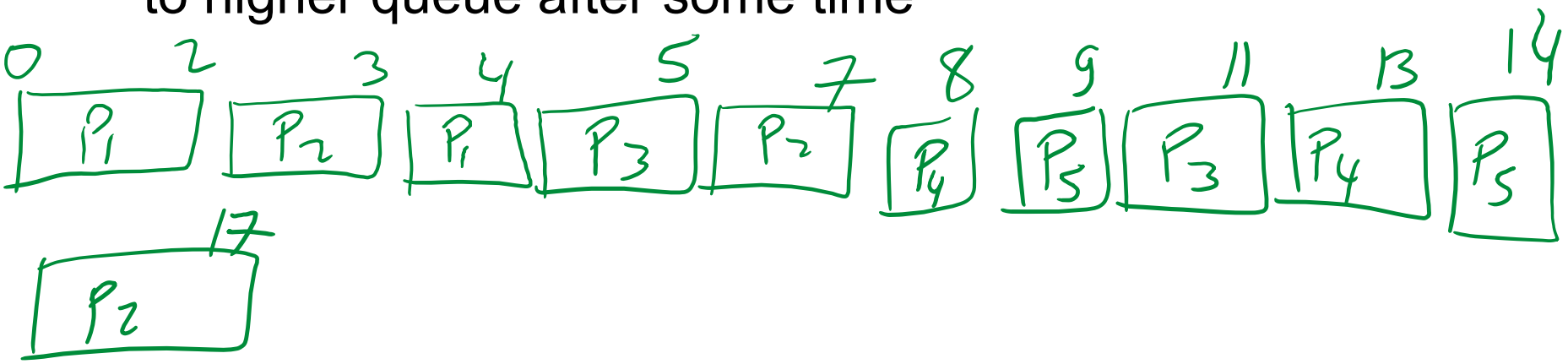
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

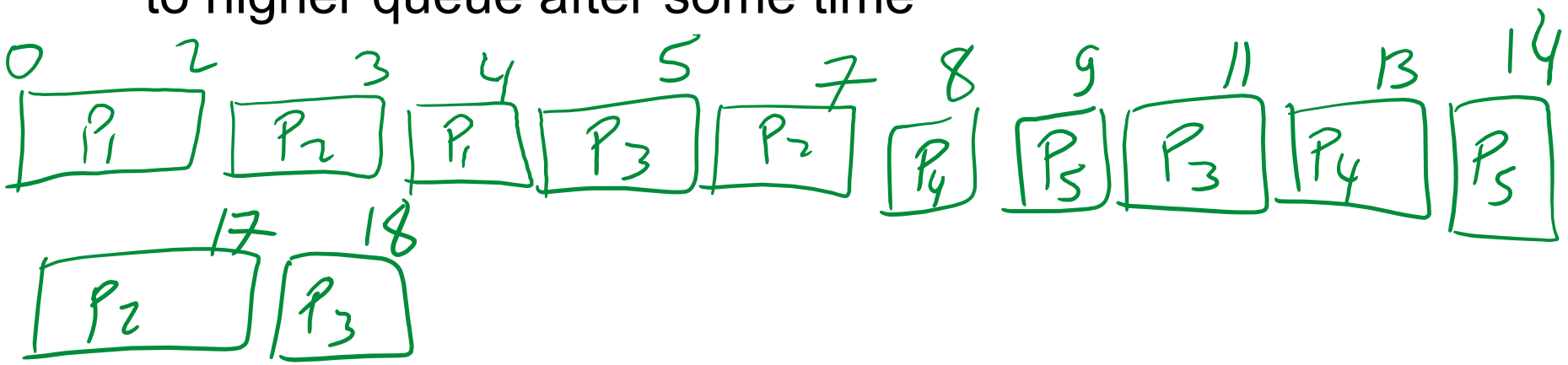
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

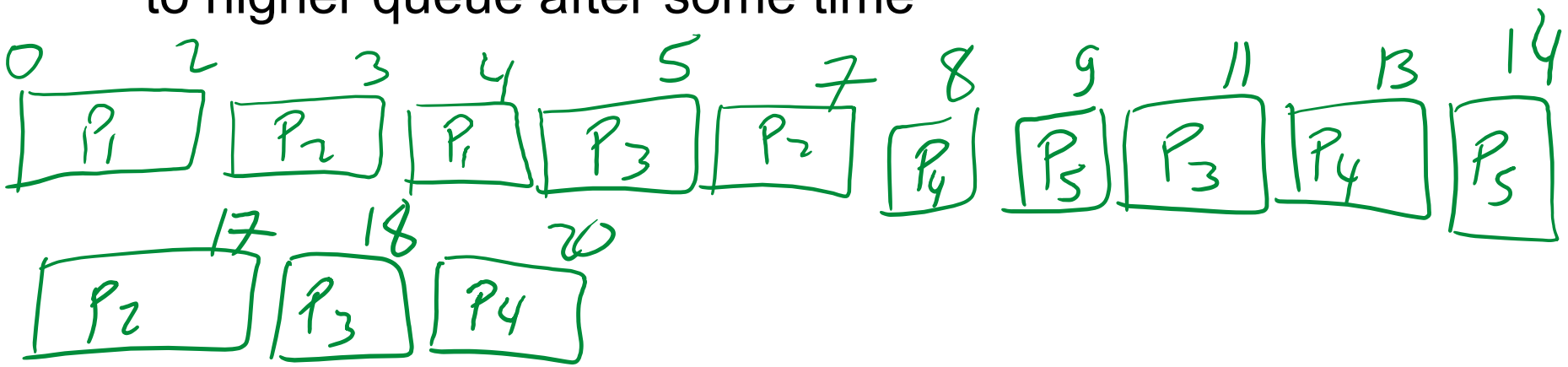
Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



Time Quantum for feedback Scheduling

- With a fixed quantum time, the turn-around time of longer processes can be high
- To alleviate this problem, the time quantum can be increased based on the depth of the queue
  - Time quantum of $RQ_i = 2^{i-1}$
- May still cause longer processes to suffer starvation.
  - Possible fix is to promote a process to higher queue after some time

Process	Arrival Time	Service Time
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2



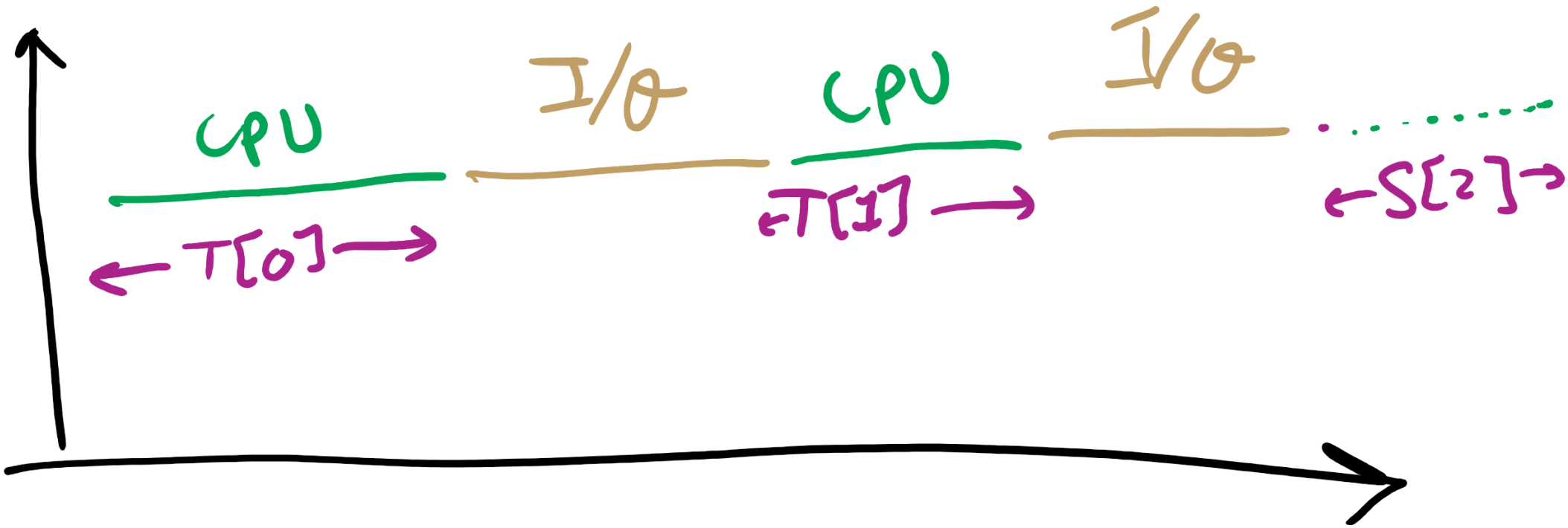
Algorithm Comparison

- Which one is the best?
- The answer depends on many factors:
  - the system workload (extremely variable)
  - hardware support for the dispatcher
  - relative importance of performance criteria (response time, CPU utilization, throughput...)
  - The evaluation method used (each has its limitations...)

Back to SJF: CPU Burst Estimation

- Let T[i] be the execution time for the ith instance of this process: the actual duration of the ith CPU burst of this process
- Let S[i] be the predicted value for the ith CPU burst of this process. The simplest choice is:
  - $S[n+1] = (1/n)(T[1]+\ldots+T[n]) = (1/n) \sum_{\{i=1 \text{ to } n\}} T[i]$
- This can be more efficiently calculated as:
  - $S[n+1] = (1/n)\, T[n] + ((n-1)/n)\, S[n]$
- This estimate, however, results in equal weight for each instance

CPU Burst Estimation



Estimating the required CPU burst

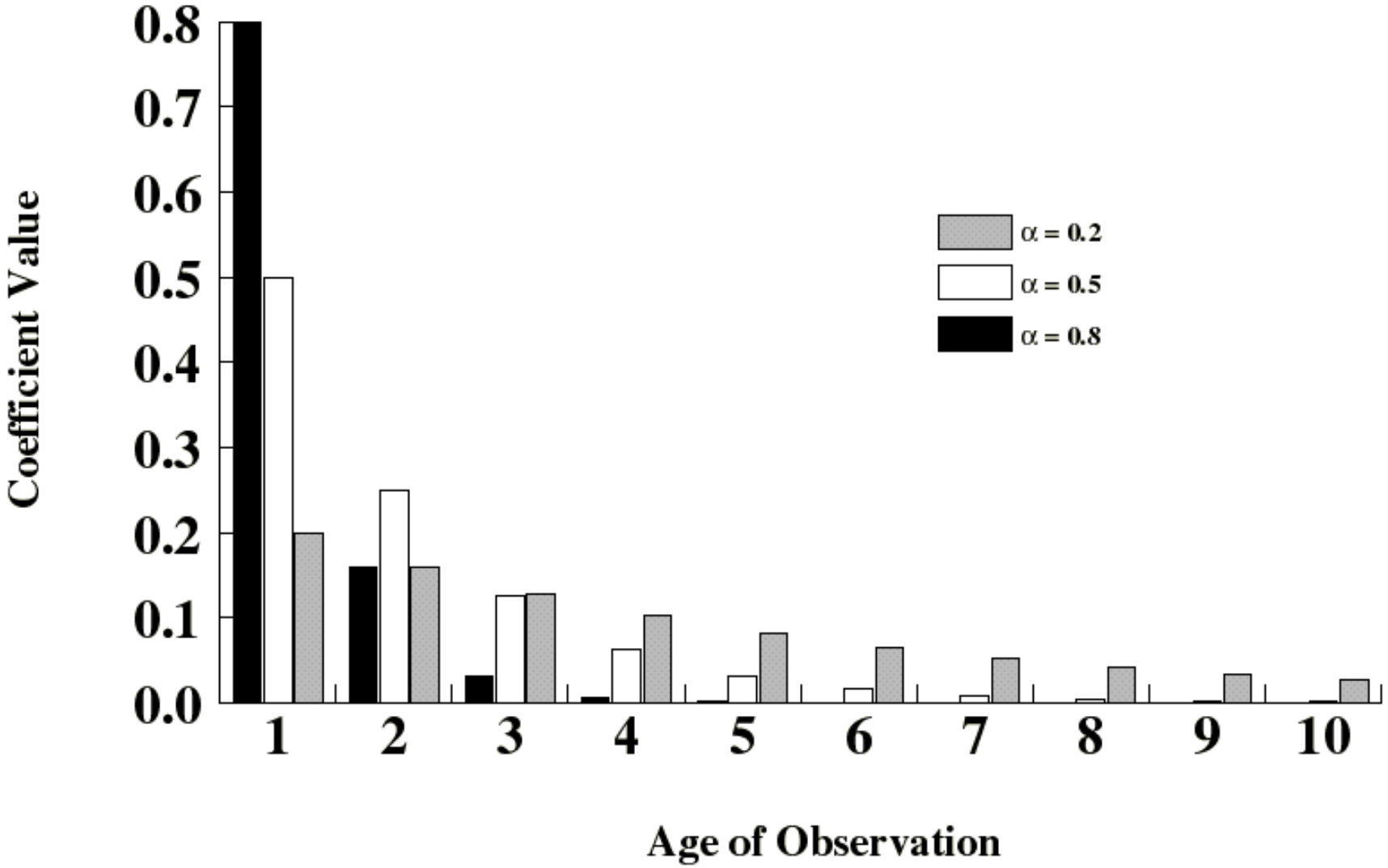
- Recent instances are more likely to better reflect future behavior
- A common technique to factor the above observation into the estimate is to use exponential averaging :

  - $S[n+1] = \alpha\, T[n] + (1- \alpha)\, S[n]$; $0 < \alpha < 1$

CPU burst Estimate (Exponential Average)

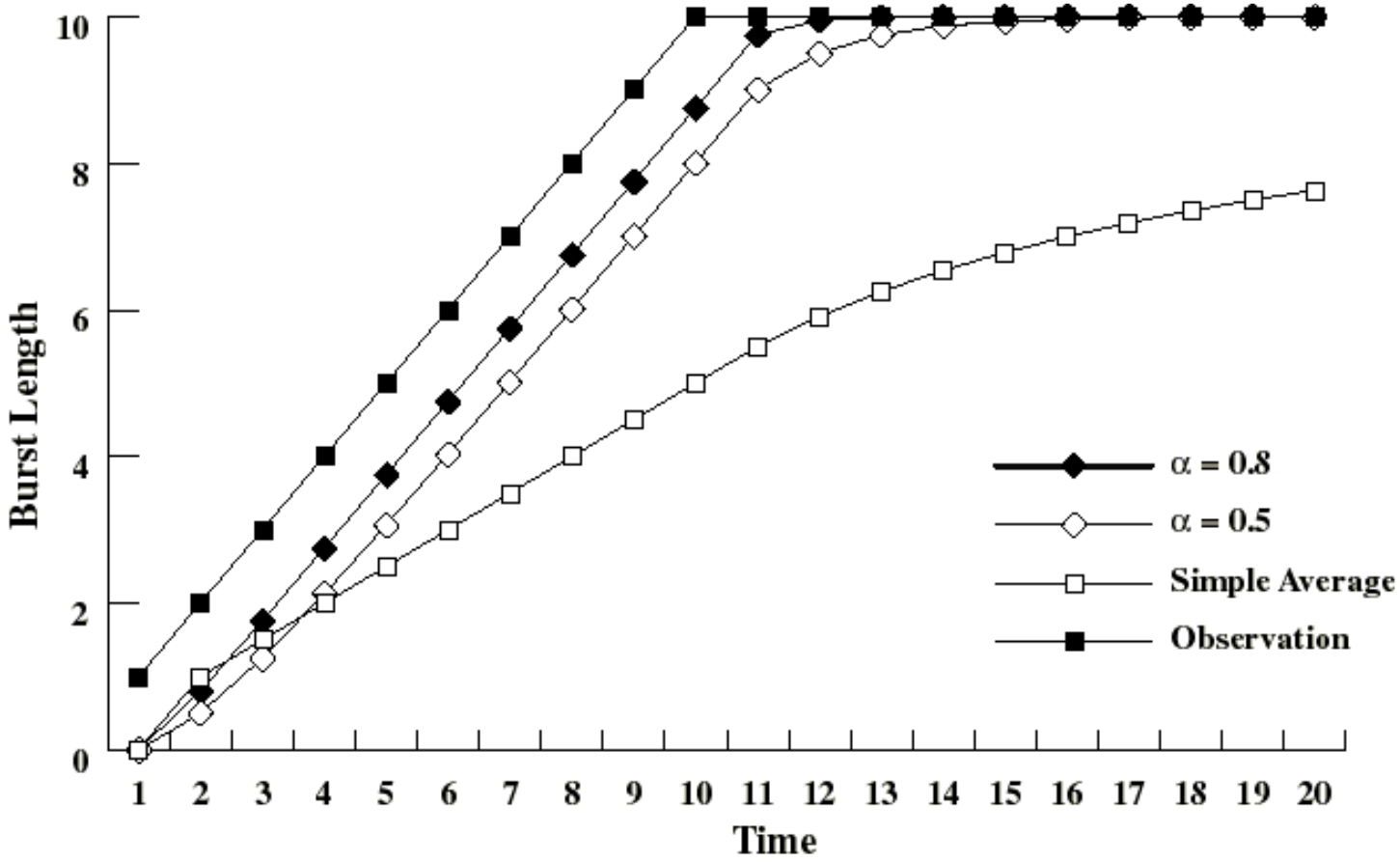
- Recent instances have higher weights, whenever α > 1/n
- Expanding the estimated value shows that the weights of past instances decrease exponentially
  - S[n+1] = α T[n] + (1- α) α T[n-1] + ... (1- α)^{i} α T[n-i] +
  - ... + (1- α)^{n}S[1]
  - The predicted value of 1st instance, S[1], is usually set to 0 to give priority to new processes

Exponentially Decreasing Coefficients

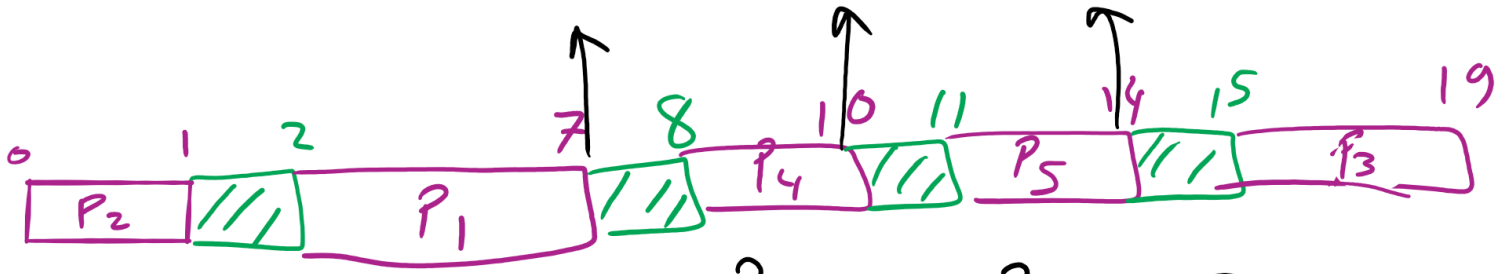


Exponentially Decreasing Coefficients

- S[1] = 0 to give high priority to new processes
- Exponential averaging tracks changes in process behavior much faster than simple averaging



FCFS Problem in HW7



$$A.R.T. = \frac{\overbrace{(7-2)}^{P_1} + \overbrace{(1-0)}^{P_2} + \overbrace{(19-8)}^{P_3} + \overbrace{(10-4)}^{P_4} + \overbrace{(14-7)}^{P_5}}{5}$$

$$= \frac{5+1+11+6+7}{5} = 6ms$$

$$CPU\ Util = \frac{19-4}{19} = \frac{15}{19}$$

CPU Burst Estimation

$$S[n+1] = \alpha T[n] + (1-\alpha) S[n]$$

$$0 < \alpha < 1$$

